Web browser

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Floriano Scioscia

Gabriele Colapinto

April 16, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.
https://creativecommons.org/licenses/by-nc/4.0/

© 2025 Gabriele Colapinto
Released under the Creative Commons Attribution–NonCommercial 4.0 International License

Contents

1 Introduction: Defining the Web Browser **5**

2 A History of Innovation and Competition **7**
2.1 The Pioneers: From Berners-Lee to Mosaic . . . 7
2.2 The Browser Wars: Netscape vs. Internet Explorer . . . 7
2.3 The Rise of Open Source and Modern Browsers . . . 8

3 The Architecture of a Modern Web Browser **11**
3.1 An Architectural Overview: The MVC Paradigm . . . 11
3.2 The User Interface (UI) . . . 12
3.3 Data Persistence . . . 12
3.4 The Browser Engine . . . 13

4 The Rendering Engine: From Code to Pixels **15**
4.1 The Role of the Rendering Engine . . . 15
4.2 The Document Object Model (DOM) . . . 16
4.2.1 Defining the DOM . . . 16
4.2.2 The DOM Tree Structure . . . 16
4.2.3 The DOM as a Programming Interface . . . 16
4.3 The Rendering Pipeline in Detail . . . 16
4.4 Rendering Engines . . . 18
4.4.1 Gecko . . . 18
4.4.2 WebKit . . . 19
4.4.3 Blink . . . 20
4.5 A Survey of Major Rendering Engines . . . 20

5 Evolution of Client-Side Programmability **23**
5.1 The Era of Plugins and Proprietary Extensions . . . 23
5.2 WebAssembly: Native-Like Performance on the Web . . . 23
5.2.1 What is WebAssembly? . . . 23
5.2.2 Goals and Security Considerations . . . 23

6 Other Core Browser Components **25**

7 Intermediaries: Optimizing the Client-Server Connection **29**
7.1 Introduction to Web Intermediaries . . . 29
7.2 Proxy-Based Browsers . . . 29
7.3 Case Study: Cloudflare as a Modern Content Delivery Network . . . 31

1 Introduction: Defining the Web Browser

Understanding the modern web browser is fundamental to engaging with nearly all web-oriented technologies. As the primary user agent for interacting with web content, the browser has evolved from a simple document viewer into a sophisticated application platform. It serves as the critical bridge between remote information stored on web servers and the end-user's device, making its architecture a key area of study for anyone involved in web development or systems design.

At its core, a web browser is a software program designed to retrieve, interpret, and display documents and other resources from the World Wide Web. It acts on behalf of a user to request content from a server—identified by a Uniform Resource Locator (URL)—and then renders that content visually, typically within a window on the user's screen. While this definition captures its primary purpose, the functionalities of a modern browser extend far beyond simple retrieval and display.

The core functionalities of a typical web browser include:

- **Resource Handling:** Displaying web resources, such as HTML pages, directly within its own window or passing specific file types to external helper applications for processing.
- **Navigation:** Retrieving resources either through the user's direct specification of a URL or by following hyperlinks embedded within other documents—the foundational mechanism of the web.
- **Session Management:** Tracking the user's navigation history to allow for easy backtracking and providing mechanisms for bookmarking pages of interest for quick recall.
- **Data Storage:** Storing user-provided information, such as values entered into web forms and account credentials for websites, to enhance user convenience and streamline subsequent interactions.
- **Accessibility:** Providing features to make web content usable for individuals with various disabilities, including blindness, low vision, and motor or hearing impairments.

These features, now standard in contemporary browsers, are the product of decades of technological evolution and competition, a history that has directly shaped the complex systems we use today.

2 A History of Innovation and Competition

To understand the architecture of modern web browsers, it is essential to appreciate their history. This evolution, marked by periods of intense competition and collaborative standardization, has directly forged the web technologies and architectural patterns in use today. From the first simple graphical viewers to today's complex application platforms, the browser's history is a story of strategic innovation that continues to define the digital landscape.

2.1 The Pioneers: From Berners-Lee to Mosaic

The genesis of the web browser is directly linked to the invention of the World Wide Web itself. In 1991, Tim Berners-Lee, while at CERN, developed the first web browser as a demonstration of the web technologies he had proposed: HTML, URLs, and HTTP. This initial program was not just a viewer; it also functioned as an HTML editor, addressing the practical need to create content for the new medium.

However, the web's explosion in popularity began with the release of NCSA Mosaic in 1993. Developed at the National Center for Supercomputing Applications, Mosaic was the first browser to allow images to be embedded directly within the text of a web page, rather than appearing in a separate window. This feature was a critical factor in transforming the web from a text-based academic tool into a visually engaging medium for the general public. The technology was later commercialized through a company called Spyglass, marking the first major step toward a commercial web.

2.2 The Browser Wars: Netscape vs. Internet Explorer

The commercial potential of the web, demonstrated by Mosaic, led directly to the first "browser war." A primary developer of Mosaic left NCSA to co-found Netscape, which released its own highly successful browser. Recognizing the strategic importance of this emerging platform, Microsoft entered the market in 1995 with Internet Explorer, a browser initially based on code licensed from Spyglass.

What followed was a period of intense competition for market dominance between Netscape and Internet Explorer. Microsoft ultimately emerged victorious, partly through its business practices but also through its technical strategy. A key tactic was to adopt the standards proposed by the World Wide Web Consortium (W3C) in an incomplete or incompatible manner while simultaneously introducing proprietary, non-standard technologies. Features like VBScript and ActiveX were designed to create vendor lock-in, encouraging developers to build web applications that worked exclusively, or best, on Internet Explorer. This approach fragmented the web and created significant challenges for cross-browser compatibility that would persist for years.

2.3 The Rise of Open Source and Modern Browsers

The end of the first browser war had a profound impact on the web's trajectory. In 1998, with its market share collapsing, Netscape made a pivotal decision: it released its browser's source code to the public, leading to the creation of the open-source Mozilla Foundation. This foundation would later develop Firefox, a standalone browser that successfully clawed back significant market share from Internet Explorer and championed web standards.

This period also saw the rise of the "software fork," where developers take an existing open-source codebase and start independent development. The web browser space provides a prime example of this pattern. The core components of the open-source Konqueror browser were forked by Apple to create WebKit, the rendering engine for its Safari browser. Later, Google forked WebKit to create Blink, the engine that now powers Google Chrome.

Google Chrome's introduction in 2008 was another transformative moment. It was architected to solve three critical problems that had emerged from the browser wars and Internet Explorer's dominance:

1. **Adherence to Web Standards:** Chrome was built with W3C standards compliance as a core principle, directly contrasting with Internet Explorer's reliance on proprietary features.
2. **Security:** Recognizing the growing threat of malicious web content, Chrome introduced a novel multi-process architecture. By running each browser tab in a separate process, it ensured that a crash or security breach in one tab would not affect the others, significantly enhancing stability and security.
3. **JavaScript Performance:** As web applications grew more complex ("Rich Internet Applications"), the performance of JavaScript, which was a purely interpreted language at the time, became a major bottleneck. Chrome introduced the V8 JavaScript engine, which featured just-in-time (JIT) compilation, dramatically increasing execution speed and enabling a new generation of powerful web applications.

Today, the browser market is dominated by Chrome and its underlying open-source project, Chromium. This dominance has, ironically, led to the re-emergence of challenges similar to those of the Internet Explorer era, such as the introduction of proprietary APIs before they are standardized. In a significant market shift, Microsoft eventually abandoned its proprietary browser architecture and rebuilt its Edge browser on the Chromium codebase, solidifying the influence of this open-source project.

This dynamic history of competition and innovation drove the need for a more structured, modular approach to browser design, leading to the sophisticated architectures found in modern browsers.

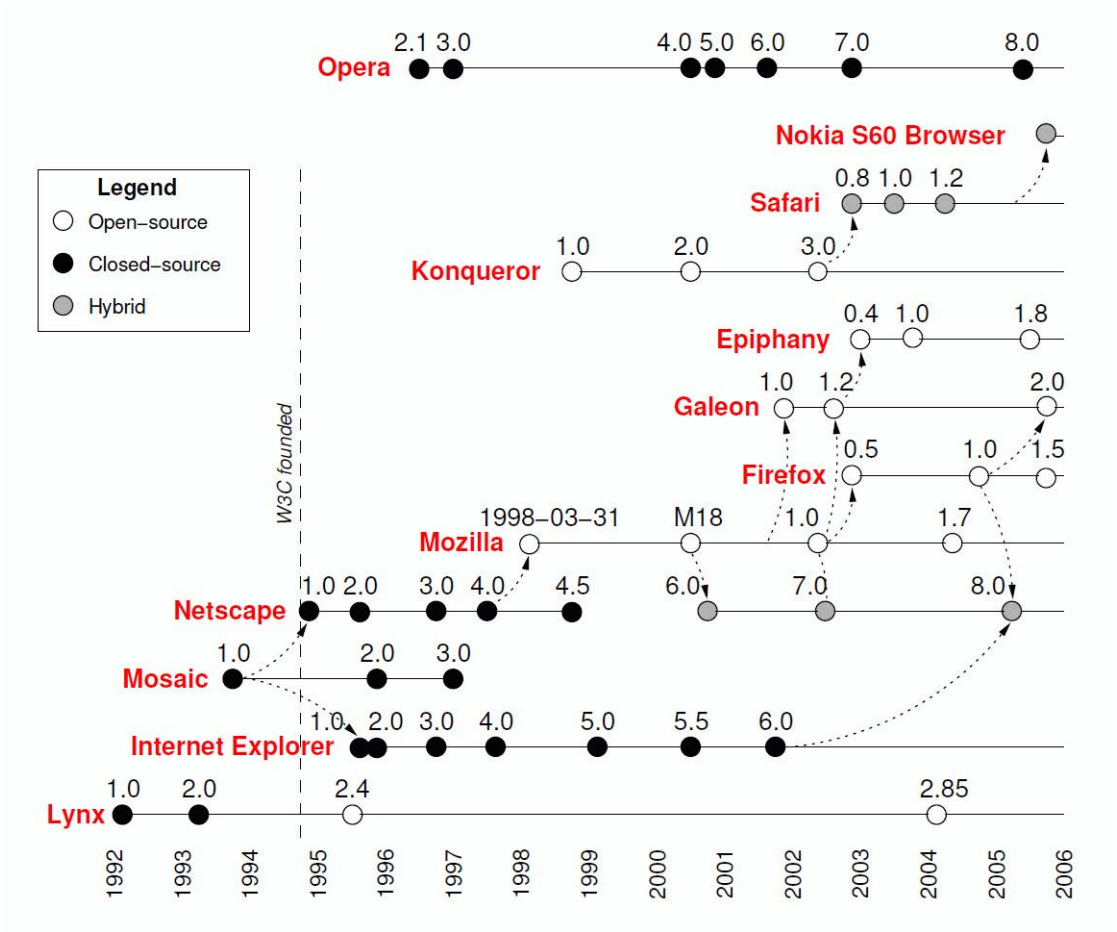

Figure 2.1: Web browser timeline

3 The Architecture of a Modern Web Browser

A modern web browser is a highly complex software system, composed of multiple interconnected components, each with a distinct responsibility. Far from being a monolithic application, its design reflects a modular architecture that can be understood using established software design patterns. This modularity allows for greater stability, security, and flexibility in adapting to the ever-changing web landscape.

3.1 An Architectural Overview: The MVC Paradigm

The general architecture of a web browser aligns well with the **Model-View-Controller (MVC)** software architectural pattern (See figure 3.1). MVC divides a program's logic into three interconnected elements, a separation that maps cleanly onto the primary functions of a browser:

- **View:** The **User Interface (UI)** is the part of the browser the user directly interacts with. It is responsible for presenting information and accepting user commands.
- **Controller:** The **Browser Engine** acts as the central coordinator. It receives input from the UI (the View) and marshals actions to the appropriate backend component, primarily the Rendering Engine.
- **Model:** The **Rendering Engine** is responsible for managing the web content itself. It parses HTML and CSS to create an in-memory representation (the Model) of the web page, which is then displayed via the UI.

Beyond these three core elements, a browser's architecture includes several other major components that work in concert to deliver a complete web experience:

- **Networking:** Handles all network communication, such as HTTP requests, to retrieve resources from servers.
- **JavaScript Interpreter:** Parses and executes JavaScript code embedded in web pages, enabling dynamic functionality.
- **Display Backend:** Provides the low-level drawing capabilities used by the Rendering Engine to paint pixels on the screen.
- **Data Persistence:** Manages the storage of user and session data on the local disk.
- **XML Parser:** Starting from a well-formed XML file it creates its tree structure representation, typically the DOM (Document Object Model). This component also check the correctness of the syntax of the document because, in case of malformed document, it returns a fatal error and the parsing fails.

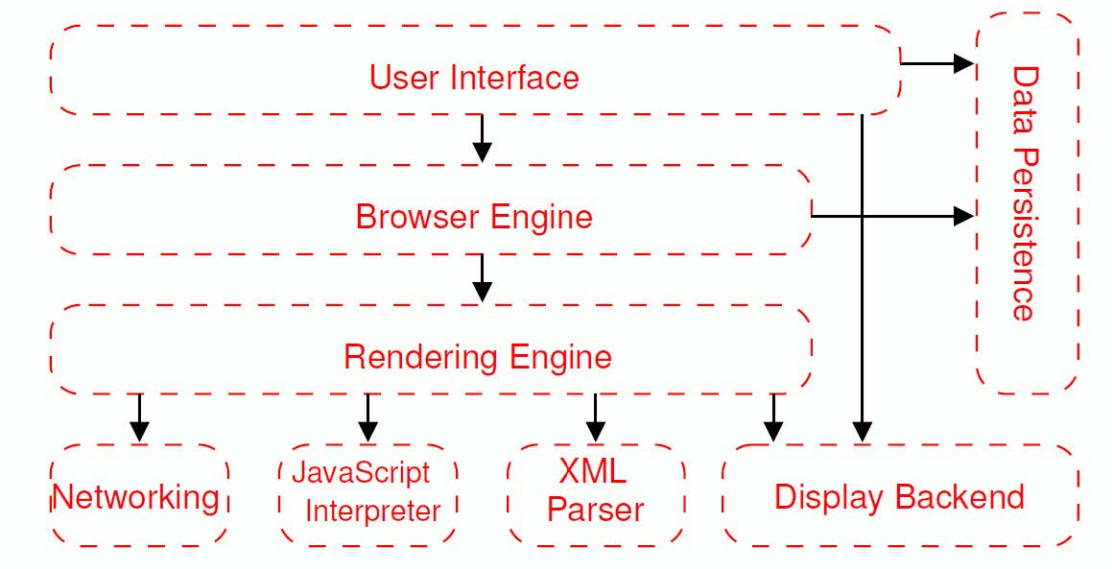

Figure 3.1: Web browser architecture

- **HTML Parser:** Considering that the XML parser requires a well-formed document and the W3C decided to use a best-effort approach to the parsing of the HTML documents, browsers have another component to parse the web pages: the HTML parser (alas not shown in figure 3.1). This component not only parses the document to create its tree and DOM representation but also performs guesswork if necessary.

3.2 The User Interface (UI)

The User Interface (a.k.a. the chrome of the browser... and yes the browser by Google was named after this component) is the visual and interactive shell of the browser. It encompasses everything the user sees and interacts with that is not the web page content itself, including the main window frame, title bar, tabs, toolbars with navigation controls (like back, forward, and reload), the address bar, and menus for bookmarks and settings.

Recent decades have seen significant innovations in browser UI design. Key advancements include **tabbed navigation**, which allows users to manage multiple web pages within a single window, and **extensions**, a concept for which Firefox is directly credited with pioneering, enabling developers to add custom functionality and enhance the browser's UI. The design of the UI is not governed by any formal specification; rather, it evolves through established best practices in user experience (UX) design and a process of competitive imitation, where successful features introduced by one browser are often adopted by others.

3.3 Data Persistence

The data persistence component is responsible for managing all user-related data that needs to be stored locally between browsing sessions. This data can be categorized into two main types:

high-level data that the user is actively aware of, and low-level data that is typically managed automatically by the browser.

Data Type	Examples
High-Level Data	Bookmarks, toolbar settings, saved form values, stored passwords.
Low-Level Data	Cookies, security certificates, cache.

Table 3.1: Data Types

This component has undergone two major innovations to improve efficiency and functionality:

1. **Embedded Databases:** Early browsers often stored data in proprietary flat-file structures. This approach was inefficient, making data access slow as the amount of information grew. Modern browsers have largely transitioned to using embedded databases like **SQLite**. This provides a more performant and open way to store and retrieve data using standard SQL queries.
2. **Cloud Synchronization:** Modern browsers now offer the ability to synchronize user data across multiple devices. Information such as bookmarks, browsing history, saved passwords, and even the list of open tabs is encrypted and stored in the cloud, allowing a user to maintain a consistent browsing environment across their desktop computers and mobile devices.

3.4 The Browser Engine

The Browser Engine serves as the core controller, coordinating operations between the User Interface and the Rendering Engine. It acts as the central hub for managing the browsing session.

Its primary functions include:

- Initiating the loading of a resource specified by a URL.
- Handling high-level browsing actions initiated by the user via the UI, such as `reload`, `back`, and `forward`.
- Monitoring and reporting the progress of page loading to the UI.

Historically, the browser engine was also responsible for managing plugins—third-party, native-code libraries like Adobe Flash and Microsoft Silverlight that could be embedded in web pages to provide advanced interactivity. However, due to significant performance and security concerns—as these plugins greatly expanded the browser's attack surface—this approach has been phased out in favor of web-native technologies like HTML5 and JavaScript. The browser engine now orchestrates these modern, more secure technologies, with its most critical task being the management of the rendering engine.

4 The Rendering Engine: From Code to Pixels

The Rendering Engine is arguably the most critical component of a web browser, as it is responsible for the visual representation of all web content. Its fundamental role is to interpret code—primarily HTML and CSS—and translate it into the formatted, interactive layout that the user sees on their screen. From text and images to complex page structures, the rendering engine transforms abstract instructions into tangible pixels.

4.1 The Role of the Rendering Engine

The primary job of the rendering engine is to parse HTML and CSS documents to calculate the layout of a web page. This involves determining the size, position, and style of every element on the page. It is important to distinguish between the full page layout and the **viewport**. The viewport is the visible area of the browser window where content is displayed. While the user only sees the content within the viewport at any given moment, the rendering engine computes the layout for the entire page, including the parts that are currently off-screen. (See figure 4.1)

A key architectural innovation, pioneered by Google Chrome, is the **multi-process architecture**. In this model, the browser runs a separate instance of the rendering engine for each open tab, each in its own isolated process. This design provides significant benefits for both stability and security. If a rendering engine instance in one tab crashes due to a bug, it does not affect the other tabs or the main browser application. Similarly, this process isolation helps contain malicious code, preventing it from accessing data in other open pages.

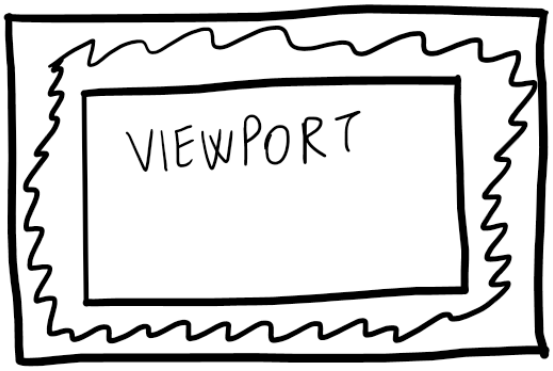

Figure 4.1: Viewport in the web page

4.2 The Document Object Model (DOM)

The Document Object Model (DOM) is the central concept and data structure upon which all modern web page rendering and interactivity are built. It is the foundational in-memory representation of a web page that allows browsers and programming languages to understand and manipulate its content.

4.2.1 Defining the DOM

The DOM is a cross-platform and language-independent programming interface for HTML and XML documents. It provides a structured, object-oriented representation of a document's contents in the browser's memory. Instead of treating the document as a simple string of text, the DOM represents it as a logical tree of nodes and objects, where each object corresponds to a part of the document. This structure allows programs to programmatically access, navigate, and modify the content, structure, and style of the document.

4.2.2 The DOM Tree Structure

The logical structure created by the browser is known as the **DOM Tree**. This is a hierarchical representation of a document's contents, where the relationships between elements are defined in a parent-child structure. In this tree, every part of the document is a **node**:

- The document itself is the root node.
- Every HTML element (e.g., `<body>`, `<h1>`, `<p>`) is an element node.
- The text contained within an element is a text node.

This tree structure accurately models the nested nature of HTML and provides a clear path for accessing any part of the document.

4.2.3 The DOM as a Programming Interface

The DOM is what enables dynamic, interactive web pages. While it is often discussed in the context of JavaScript, the DOM itself is not part of the JavaScript language. Instead, it is a Web API that JavaScript can use to interact with a web page. JavaScript acts as the scripting language that can "talk" to the DOM, using the API to perform actions such as:

- Changing the text content of an element.
- Adding or removing elements from the page.
- Modifying the CSS styles of an element.

This powerful interface between a scripting language and the document's structure is the engine of all modern client-side web application functionality.

4.3 The Rendering Pipeline in Detail

The process of turning HTML and CSS code into visible pixels on the screen can be understood as a sequential pipeline executed by the rendering engine (See figure 4.2). This process involves several distinct stages:

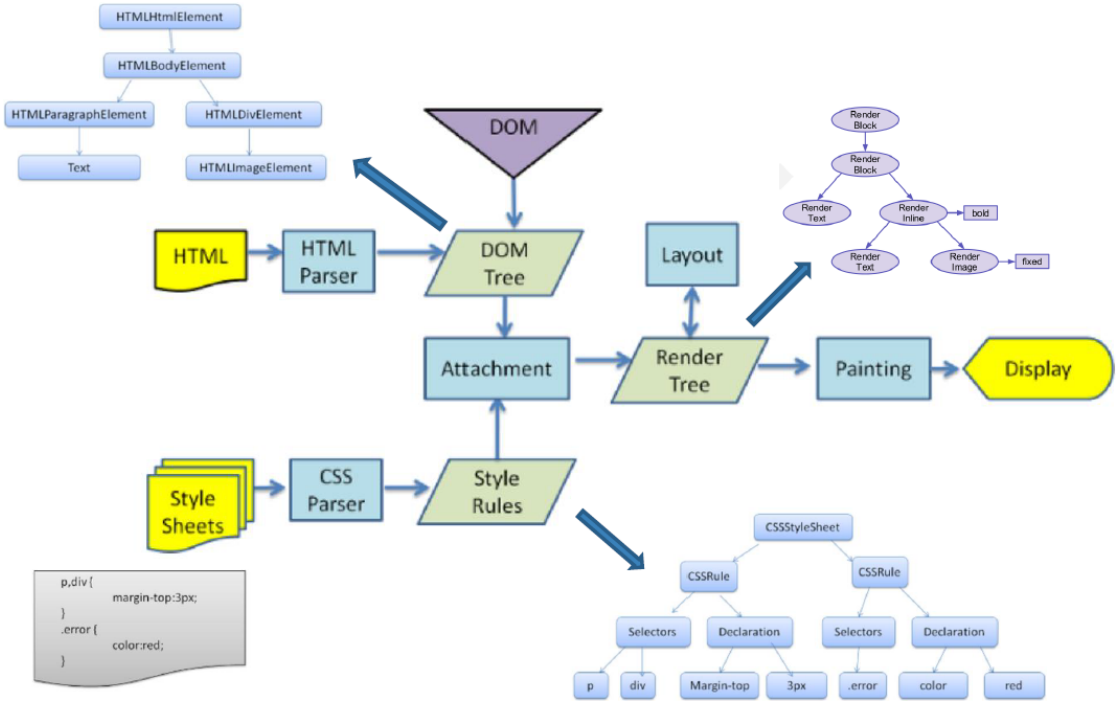

Figure 4.2: Rendering engine basic flow

1. **HTML Parsing:** The process begins with the HTML parser, which processes the HTML markup from the server. As it parses the document, it constructs the **DOM Tree** in memory, creating the logical representation of the page's structure.

2. **CSS Parsing:** Simultaneously, the CSS parser processes all associated stylesheets, including external CSS files and inline style tags. It parses this CSS into a data structure representing the style rules, which maps selectors to declarations of visual properties (e.g., color, size, position). This structure is then used in the next stage to style the document's elements.

3. **Attachment/Render Tree Construction:** In this stage, the DOM tree and the style rules are combined. The engine traverses the DOM tree and, for each visible node, attaches the relevant style rules. The result is a new tree called the **Render Tree** (See figure 4.3), which contains only the visual information needed to display the page.

4. **Layout:** The layout stage takes the render tree and calculates the exact coordinates for each node. This process, also known as "reflow," determines the precise position and size of every element within the viewport and the page as a whole.

5. **Painting:** The final stage is painting. The rendering engine traverses the render tree and issues painting commands to a **display backend**. This backend component is responsible for using the operating system's native graphical libraries to draw the pixels on the screen, bringing the webpage to life.

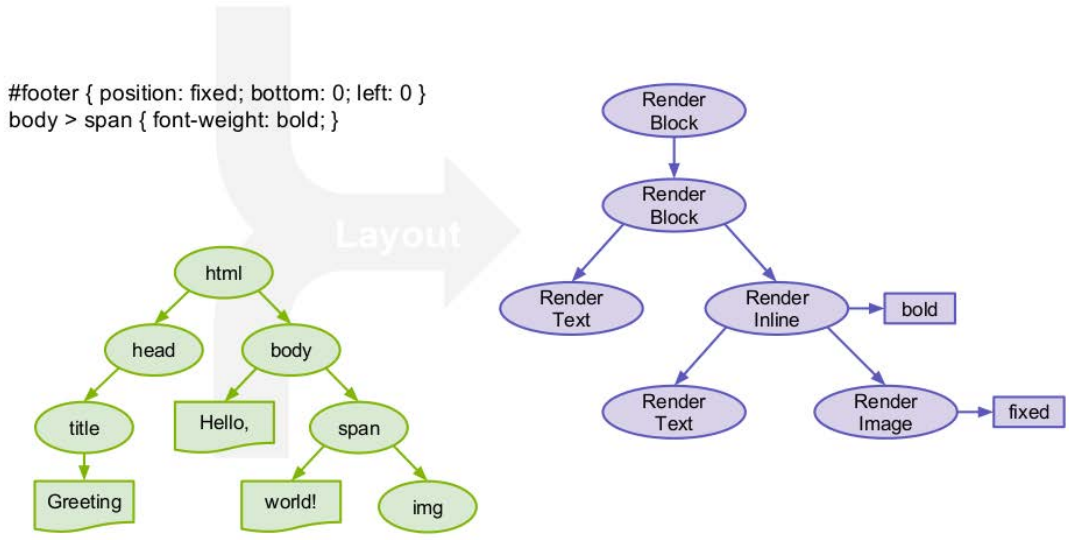

Figure 4.3: Render tree

4.4 Rendering Engines

A rendering engine is the core component of a web browser responsible for interpreting web content (HTML, CSS, XML) and transforming it into a visual representation displayed on the screen. Different browsers adopt different rendering engines, each with its own internal architecture and design choices.

4.4.1 Gecko

Gecko is the rendering engine developed by Mozilla and used primarily in Firefox-based browsers. It is designed to be modular and standards-compliant, supporting a wide range of web technologies.

The main components of the Gecko rendering engine include:

- **Style System:** Responsible for parsing CSS stylesheets and computing the final styles applied to DOM elements. It retrieves CSS data through the networking component (Necko).
- **Image Library:** Manages the loading and decoding of image resources. It interacts with Necko to fetch image data from the network.
- **Content Model:** Represents the internal structure of web documents and coordinates interactions among the various Gecko components.
- **Frame Constructor:** Builds the frame tree by combining DOM structure and computed styles. This step prepares all the necessary information before sending the rendered page to the display backend.

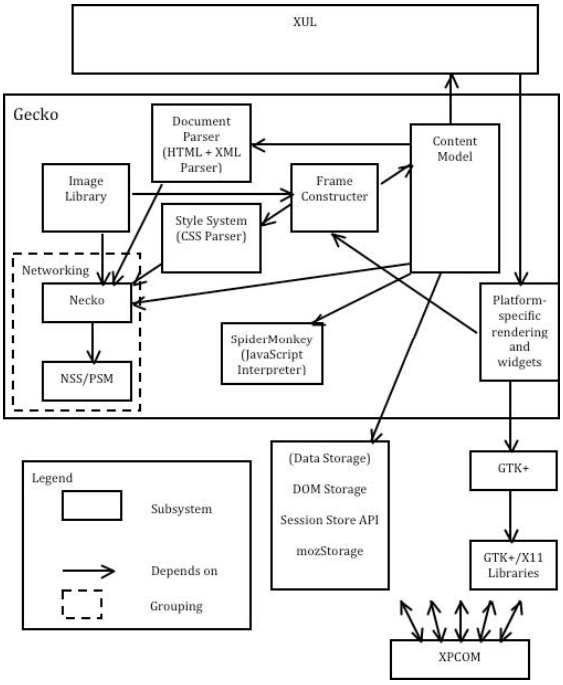

Figure 4.4: Gecko rendering engine architecture

Gecko emphasizes strict standards compliance and separates concerns such as networking, parsing, layout, and rendering into well-defined subsystems.

4.4.2 WebKit

WebKit is an open-source rendering engine originally developed by Apple and later adopted by several browsers and platforms, including Safari and early versions of Chrome. It provides both rendering capabilities and a JavaScript engine.

The architecture of WebKit is composed of the following main elements:

- **WebKit Embedding API:** Acts as an interface between the rendering engine and the browser user interface, allowing WebKit to be integrated into different applications.

- **WebCore:** Contains the core application logic of the rendering engine. It handles resource loading, HTML and CSS parsing, layout computation, style resolution, painting, event handling, editing features, and bindings to JavaScript.

- **JavaScript Engine:** WebKit can use either JavaScriptCore or, in some implementations, V8. The JavaScript engine parses and executes scripts and allows manipulation of the DOM.

WebKit has been widely ported to multiple platforms, each using different graphics, networking, font, and JavaScript subsystems depending on the target environment.

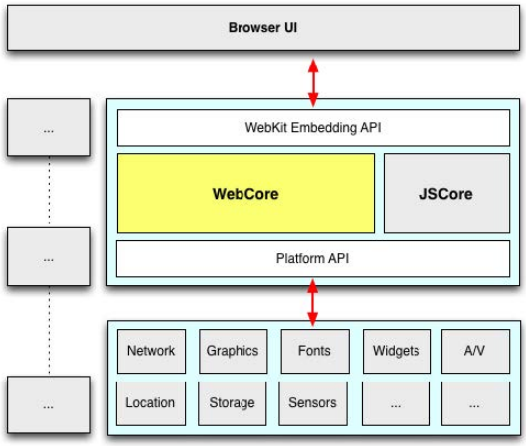

Figure 4.5: WebKit rendering engine architecture

4.4.3 Blink

Blink is a rendering engine developed as part of the Chromium project. It was created as a fork of WebKit's WebCore component and is currently used by many modern browsers.

Blink is adopted by:

- Google Chrome (from version 28)
- Opera (version 15 and later)
- Microsoft Edge (from 2019)
- Amazon Silk
- Chromium-based browsers
- Android WebView (from Android 4.4)
- Qt WebEngine

As a fork of WebKit, Blink retains a similar high-level structure but evolves independently. It focuses on performance optimization, security improvements, and tighter integration with the Chromium multi-process architecture.

Blink works closely with the V8 JavaScript engine and the Chromium networking stack, enabling efficient execution of web applications and consistent behavior across platforms.

4.5 A Survey of Major Rendering Engines

Different browsers use different rendering engines, though many modern browsers share engines due to open-source collaboration and software forking. Table 4.1 summarizes the major rendering engines in use today.

It is noteworthy that many of these engines share a common lineage. **Blink**, the engine for Chrome, is a fork of **WebKit**, which is used by Safari. WebKit, in turn, was originally a fork of

Engine	Associated Browser(s)	Open Source
Blink	Chrome 28+, Edge (2019+), Opera 15+	Yes
Gecko	Firefox	Yes
WebKit	Safari, Chrome (pre-28), Android Browser	Yes
Trident	Internet Explorer (Legacy)	No
KHTML	Konqueror	Yes

Table 4.1: Major Rendering Engines

KHTML, the engine for the Konqueror browser. This history highlights the importance of open-source development in the browser ecosystem. With the page rendered, the next consideration is how browsers execute the advanced client-side logic that defines modern web applications.

5 Evolution of Client-Side Programmability

The web has undergone a strategic transformation from a repository of static, hyperlinked documents to a platform for rich, interactive applications. This shift has been driven by the evolution of technologies designed to execute complex code directly on the client-side—within the browser itself. This progression has moved from proprietary, insecure plugins to a standardized, high-performance virtual machine built for the modern web.

5.1 The Era of Plugins and Proprietary Extensions

In the early days of the interactive web, browsers relied on third-party technologies to provide functionality beyond what HTML and early JavaScript could offer. Browser **plugins** like Adobe Flash and Microsoft Silverlight, along with Microsoft's **ActiveX** technology, served this purpose. They allowed developers to embed pre-compiled, native code components directly into web pages to enable features such as complex animations, video playback, and rich application interfaces.

However, these technologies were beset with problems that ultimately led to their decline. The primary reasons for their deprecation were significant **performance and security concerns**. By embedding external, native code, these plugins expanded the attack surface of the browser, creating numerous vulnerabilities that could be exploited by malicious actors. They were also resource-intensive, particularly on emerging mobile devices, leading to poor performance and battery life. As a result, the industry moved to phase them out in favor of safer, web-native standards.

5.2 WebAssembly: Native-Like Performance on the Web

The modern, standardized successor to the plugin model is **WebAssembly (Wasm)**. It was created to solve the same fundamental problem—enabling high-performance applications on the web—but in a way that is secure, efficient, and part of the open web platform.

5.2.1 What is WebAssembly?

WebAssembly is a binary instruction format for a stack-based virtual machine that runs inside the browser. It is not a programming language meant to be written by hand; rather, it is a portable **compilation target**. This means that code written in high-performance languages like C, C++, and Rust can be compiled into a compact Wasm binary format. This binary can then be downloaded and executed by the browser at near-native speed, far surpassing the performance limitations of even highly optimized JavaScript for CPU-intensive tasks.

5.2.2 Goals and Security Considerations

WebAssembly was designed with several core goals in mind to address the shortcomings of previous technologies:

- **Efficient and Fast:** To be encoded in a size-efficient binary format for fast downloading and to execute at near-native speed by leveraging common hardware capabilities.
- **Safe:** To run within a memory-safe, sandboxed execution environment inside the browser. This sandboxing is critical, as it enforces the browser's security policies and prevents Wasm code from accessing arbitrary system resources.
- **Open and Debuggable:** To be a fully open and standardized part of the web platform. It also includes a human-readable text format (`.wat`) that aids in debugging, testing, and understanding the compiled code.

Despite its powerful capabilities, WebAssembly has not yet achieved universal adoption for all use cases. A primary reason is that, like the plugins it was designed to replace, introducing a new execution environment inherently increases the attack surface of the browser. Although designed with a strong security model, the potential for discovering new vulnerabilities in its implementation remains a concern, leading to a cautious but steady adoption for specific, high-performance applications.

6 Other Core Browser Components

Beyond the rendering engine, several other critical components are essential for a complete and functional browsing experience.

- **JavaScript Interpreter:** This component is responsible for parsing and executing JavaScript code embedded in a webpage. Its primary function is to enable dynamic page behavior, most notably through DOM manipulation—adding, removing, or modifying elements and styles in real-time. Historically, pure interpreters were slow, as they had to process the same instructions repeatedly within loops or functions. Modern browsers have overcome this limitation by incorporating a **Just-In-Time (JIT) Compiler**. A JIT compiler identifies sections of code that are executed frequently and compiles them into faster native machine code at runtime (See figure 6.1). This approach dramatically improves the performance of complex web applications. Key JIT-enabled JavaScript engines include **V8** (Chromium), **SpiderMonkey** (Firefox), and **Nitro** (Safari).
- **Networking:** The networking component handles all network communication for the browser. It is responsible for retrieving resources from URLs using protocols like HTTP and, in some cases, FTP. Its duties extend to managing the presentation layer of the OSI stack, where it translates between different character sets and resolves MIME media types to understand the format of incoming files. This component also manages a local cache of recently retrieved resources to minimize network traffic and speed up subsequent page loads.
- **Display Backend:** The display backend acts as an interface to the operating system's graphical primitives, providing the tools needed to draw windows, UI widgets, and fonts. A significant modern innovation in this area is **hardware acceleration**. In this model, each render object in the render tree is treated as a separate graphic layer, which is then uploaded to the Graphics Processing Unit (GPU) as a texture. The GPU can then composite these layers and perform transformations (like resizing or rotating) with high performance and minimal impact on the CPU. While powerful, this approach has a potential drawback on resource-constrained mobile devices, where loading an excessive number of textures into the GPU's dedicated memory can lead to performance issues or even crashes.
- **XML Parser:** This is a dedicated parser for processing XML documents. Its primary importance in modern web development is enabling the **AJAX (Asynchronous JavaScript and XML)** programming paradigm. AJAX allows a webpage to make additional HTTP requests to a server to fetch data (often, but not exclusively, in XML format) without requiring a full page reload. This data can then be used to dynamically update parts

Figure 6.1: JIT compiles part of the code into faster native machine code

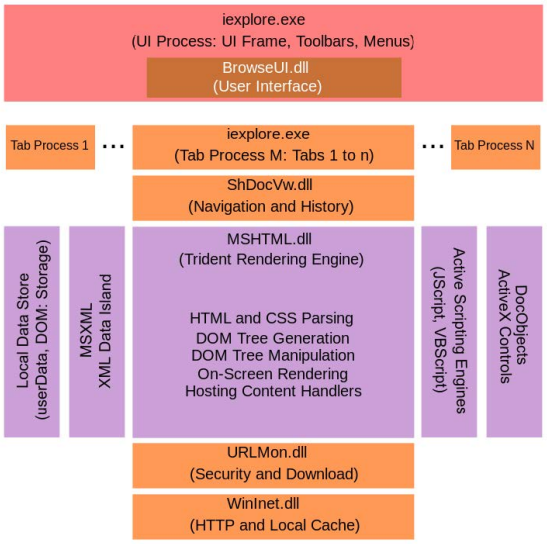

Figure 6.2: Internet Explorer architecture

of the page, creating the seamless, interactive experience characteristic of Rich Internet Applications.

Figures 6.2, 6.3 and 6.4 show the architecture of the major modern web browsers which contain the elements exposed in the preceding sections.

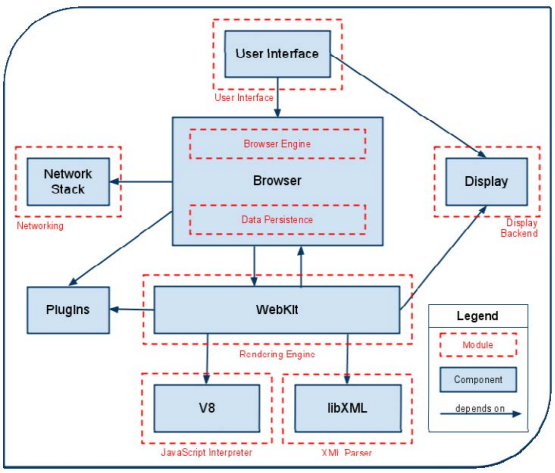

Figure 6.3: Google Chrome architecture

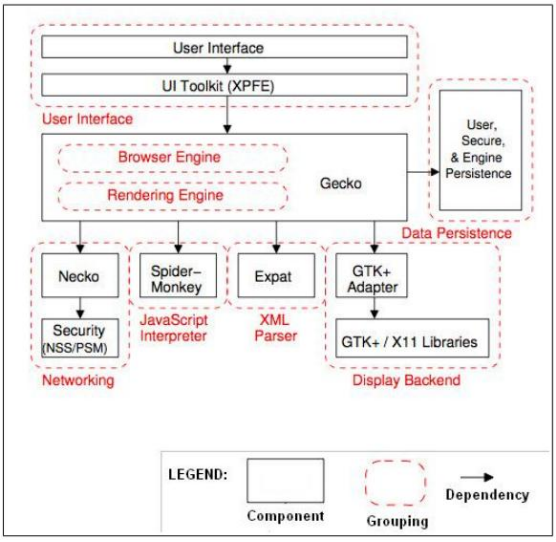

Figure 6.4: Firefox architecture

7 Intermediaries: Optimizing the Client-Server Connection

7.1 Introduction to Web Intermediaries

While the client-server model is fundamental to the web, the direct communication channel between a browser and an origin server can be augmented by intermediary services. These systems, such as specialized proxy servers and Content Delivery Networks (CDNs), sit between the client and server. They are strategically designed to improve the web experience by enhancing performance, bolstering security, and increasing the scalability of web applications.

7.2 Proxy-Based Browsers

Proxy-based web browsers represent a specific type of intermediary architecture designed to serve web content to resource-constrained devices, such as the first generations of smartphones. (See figure 7.1)

In this model, the client device does not communicate directly with the origin server. Instead, the request lifecycle is as follows:

1. The client browser sends a page request to the browser vendor's proxy server using a proprietary, optimized protocol.
2. The proxy server fetches all necessary resources (HTML, CSS, images) from the origin server over a high-bandwidth connection.
3. The proxy renders the complete page, compresses the output, and encodes it into a single, proprietary resource.

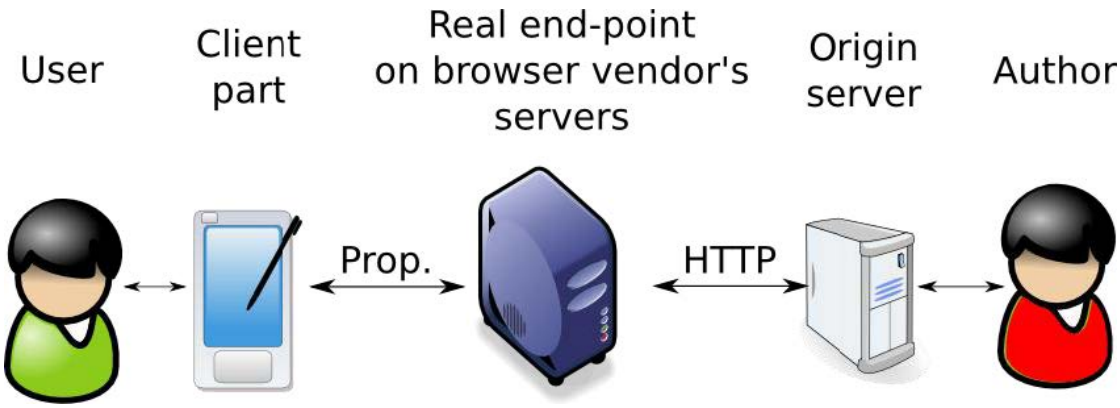

Figure 7.1: Proxy based browser infrastructure

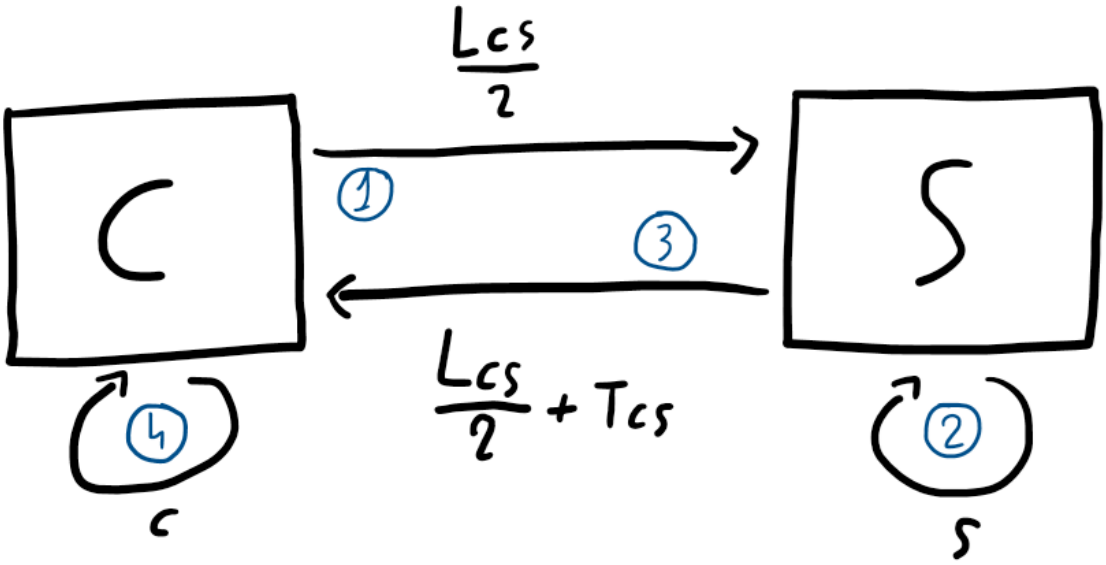

Figure 7.2: Client-server turnaround – The sequence of steps is highlighted in blue.

4. This optimized resource is sent back to the client device, which can process it much more efficiently than the raw source files.

The performance benefits of this model can be analyzed by comparing the formulas for **turnaround time** (T, the total time to load and render a page):

- **Standard Browser:** $T = L_{cs} + T_{cs} + S + C$ (See figure 7.2)
  - L_{cs}: Latency between client and source server. (Referring to the empty client request and server response – In this text we use L_{cs} to denote the whole latency. We could have used a different notation to refer to the latency of the single messages and, in such case, we would have had $2 \cdot L_{cs}$ in the formula.)
  - T_{cs}: Data transfer time from source to client (Referring to the payload of the server response).
  - S: Server processing time.
  - C: Client processing time (parsing and rendering).
- **Proxy-Based Browser:** $T' = L_{cp} + L_{ps} + T_{cp} + T_{ps} + P + S + C'$ (See figure 7.3)
  - L_{cp}, L_{ps}: Latency for client-proxy and proxy-source connections (Referring to the empty requests and responses).
  - T_{cp}, T_{ps}: Data transfer times for client-proxy and proxy-source (Referring to the payload of the responses).
  - P: Proxy server processing time.
  - C': Client processing time for the pre-encoded format.

The proxy model aims to minimize the total time T' by optimizing each component:

- Geographically distributed proxy servers reduce overall latency (L) by reducing the latency for client-proxy and proxy-source connection with respect to the latency for the direct client-server connection ($L_{cp}, L_{ps} \ll L_{cs}$).

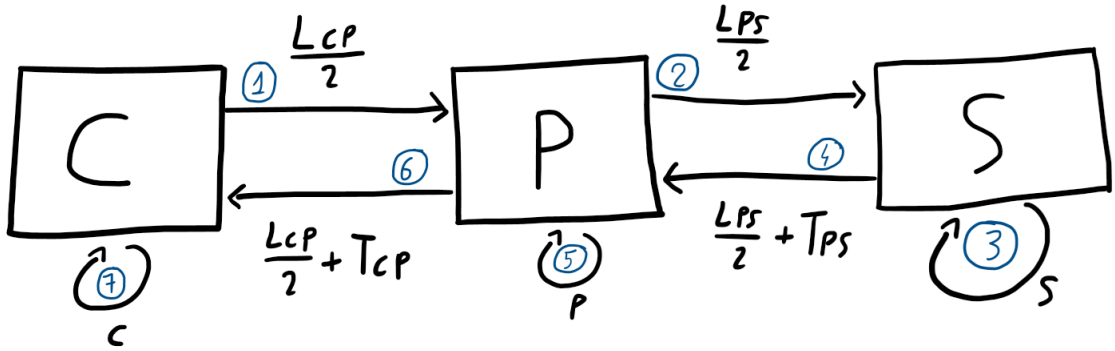

Figure 7.3: Client-proxy-server turnaround – The sequence of steps is highlighted in blue.

- The proxy's high-bandwidth connections and caching reduce the server-to-proxy transfer time with respect to the direct client-server transfer time ($T_{ps} \ll T_{cs}$).
- Data compression significantly reduces the proxy-to-client transfer time with respect to the direct client-server transfer time ($T_{cp} \ll T_{cs}$). For instance, a page composed of ten resources totaling 1 MB might be consolidated and compressed by the proxy into a single resource of just 100 kB, drastically reducing T_{cp} relative to T_{cs}.
- Pre-encoding the page into a more rapidly processable format reduces the client-side processing effort ($C' < C$).
- The proxy processing time (P) is small because the proxy has great computational power.

7.3 Case Study: Cloudflare as a Modern Content Delivery Network

Cloudflare is a prominent modern example of a web intermediary, operating at a massive scale. It provides a suite of services including a **Content Delivery Network (CDN)**, a **reverse proxy**, and comprehensive cybersecurity. As of January 2025, Cloudflare's security services protect approximately 20% of all websites on the internet.

Cloudflare's architecture extends the core principles of proxy-based browsers—optimizing the path between the user and the content—to any website on a global scale. Its key offerings can be categorized as follows:

- **Performance Enhancement:** Cloudflare's vast global network acts as a CDN, caching static content in data centers physically closer to end-users. This practice, known as **Edge Computing**, dramatically reduces latency and accelerates page load times. Services like `Cloudflare Workers` even allow developers to run serverless code directly at the edge, further minimizing delays.
- **Security and Protection:** As a reverse proxy, Cloudflare sits in front of the origin server and absorbs malicious traffic before it can cause harm. It is a leading provider of **DDoS mitigation**. In February 2023, for example, Cloudflare reported blocking the largest HTTP DDoS attack on record, which peaked at 71 million requests per second,

demonstrating the sheer scale of its protective capabilities. Its **"Project Galileo"** initiative provides these vital protection services for free to at-risk public interest groups, journalists, and activists. Additionally, its **Web Application Firewall (WAF)** and bot management services provide a crucial layer of security against application-level threats.

The architecture of the modern web is thus a dynamic ecosystem defined by the holistic interaction of its three core pillars: sophisticated client-side applications like web browsers that render complex content; highly configurable server-side platforms like Apache that provide that content with precision; and powerful network intermediaries like Cloudflare that optimize and secure the connection between them. Together, these components deliver the high-performance, resilient, and secure user experience that defines today's internet.